



NeuralForgeAI

Distributed YOLO Training Cluster

A Comprehensive Technical Guide for
Production-Ready Distributed Machine Learning
March 17, 2026

Docker + Celery + Optuna + Redis + PostgreSQL

Author

William Rodriguez

Líder de IA y Arquitecto de Soluciones

eCaptureDtech

Badajoz, Extremadura, España

Contact Information:

LinkedIn: es.linkedin.com/in/wisrovi-rodriguez

Email: william.rodriguez@ecapturedtech.com

GitHub: github.com/wisrovi

Expert in MLOps and Cloud Architecture

Executive Summary

The NeuralForgeAI Distributed YOLO Training Cluster is a comprehensive ecosystem designed to orchestrate distributed machine learning workloads for training YOLO models with hyperparameter optimization. This system leverages modern technologies including Celery for task orchestration, Optuna for hyperparameter tuning, Docker for container isolation, and Redis as a message broker. The architecture implements a microservices pattern where each component plays a specific role: the Control Server provides REST and Gradio interfaces, the Manager coordinates Optuna studies, the Invoker executes Docker containers for training, and the Worker Executor performs the actual YOLO training. This document provides complete technical documentation including architecture analysis, installation guides, configuration examples, use cases, and best practices for deploying and maintaining the system.

Contents

Executive Summary	1
Contents	2
List of Figures	3
List of Tables	4
Listings	5
1 Introduction	6
1.1 Background and Motivation	7
1.2 Project Overview	7
1.3 Objectives of This Document	7
2 Technical Architecture	9
2.1 System Architecture Overview	10
2.2 Component Architecture	11
2.2.1 Control Server	11
2.2.2 Manager (Optuna Orchestrator)	11
2.2.3 Invoker (Celery Worker)	11
2.2.4 Worker Executor (Docker Container)	12
2.2.5 NeuralForgeAI Frontend	12
2.3 Data Flow and Message Passing	12
2.4 Database Schema	12
3 Installation Guide	14
3.1 System Requirements	15
3.2 Infrastructure Setup	15
3.3 Control Server Deployment	15
3.4 Manager Deployment	15
3.5 Worker Deployment	16
3.6 NeuralForgeAI Frontend Deployment	16
4 Configuration Guide	17
4.1 Configuration File Structure	18
4.2 Configuration Parameters	19
4.3 Worker Configuration	19
5 Usage Examples	21
5.1 Launching a Training Study via API	22
5.2 Launching a Training Study via Gradio	22
5.3 Launching a Training Study via NeuralForgeAI	23
5.4 Querying Available Workers	23
5.5 Managing Queued Tasks	23

6	Practical Use Cases	25
6.1	Use Case 1: Image Classification Model Training	26
6.2	Use Case 2: Object Detection for Autonomous Vehicles	26
6.3	Use Case 3: Industrial Quality Inspection	26
7	Metrics and Performance	27
7.1	System Metrics	28
7.2	Expected Performance Characteristics	28
7.3	Monitoring Tools	28
8	Best Practices and Troubleshooting	29
8.1	Deployment Best Practices	30
8.2	Common Issues and Solutions	30
9	Conclusions and Future Work	31
9.1	Conclusions	32
9.2	Future Work	32
	Bibliography	33
	Bibliography	33
A	Advanced Configurations	34
A.1	Advanced Search Space Configurations	35
A.2	Docker Resource Limits	35
A.3	Multi-Cluster Deployment	36
B	API Reference	37
B.1	REST API Endpoints	38
C	Acknowledgments	39

List of Figures

2.1 System Architecture Overview	10
--	----

List of Tables

Listings

3.1	Deploy Control Server	15
3.2	Deploy Manager	15
3.3	Deploy GPU Worker	16
3.4	Deploy NeuralForgeAI	16
3.5	Frontend Development	16
4.1	Example Training Configuration	18
4.2	Worker Configuration	19
5.1	Launch Training via API	22
5.2	Check Study Status	22
5.3	List Available Workers	23
5.4	List Queued Tasks	23
5.5	Revoke Task	23
5.6	Reprioritize Task	23
A.1	Advanced Search Space	35
A.2	Advanced Docker Configuration	35

Capítulo 1

Introduction

1.1 Background and Motivation

The training of deep learning models, particularly YOLO (You Only Look Once) models for computer vision tasks, requires significant computational resources and proper orchestration. Traditional single-machine training approaches face limitations in scalability, resource utilization, and hyperparameter optimization efficiency. The NeuralForgeAI project addresses these challenges by implementing a distributed training cluster that can scale across multiple GPU nodes while maintaining efficient resource management and enabling automated hyperparameter optimization.

The system was designed with the following key objectives: first, to provide a scalable infrastructure capable of training YOLO models across multiple GPU nodes; second, to automate hyperparameter optimization using Optuna's intelligent search algorithms; third, to ensure complete isolation between training executions through Docker containerization; fourth, to offer multiple interfaces including REST API, Gradio UI, and React frontend for different user preferences; and finally, to implement priority-based task scheduling for optimal resource allocation.

The distributed architecture allows organizations to leverage their GPU infrastructure more efficiently, with each worker node capable of running multiple training trials in isolation. The integration with Optuna enables automated hyperparameter optimization, significantly reducing the time and expertise required to find optimal model configurations.

1.2 Project Overview

The NeuralForgeAI cluster consists of five main repositories that work together to provide a complete training solution:

The Control Server repository serves as the entry point to the system, providing a FastAPI REST API for programmatic access and a Gradio interface for visual interaction. This component handles user authentication, configuration validation, and task submission to the underlying orchestration layer.

The Manager repository contains the Optuna orchestration logic that coordinates hyperparameter optimization studies. It manages the creation of studies, suggests hyperparameters for each trial, and collects results to determine optimal configurations.

The Invoker repository implements the Celery worker that listens for training tasks and manages their execution. It creates temporary workspaces, launches Docker containers for isolated training, and collects results upon completion.

The Worker repository provides the Docker executor container that performs the actual YOLO training. This ephemeral container receives configuration, executes the training process, logs metrics to MLflow, and returns results.

Finally, the NeuralForgeAI repository contains a modern React frontend that provides an intuitive user interface for launching training studies and monitoring their progress.

1.3 Objectives of This Document

This technical documentation aims to provide comprehensive coverage of all aspects of the NeuralForgeAI system. The document presents the complete architecture, detailing how each component interacts with others to form a cohesive distributed training platform. Installation procedures are documented with step-by-step instructions for deploying each

component. Configuration examples demonstrate how to customize the system for different use cases. Practical use cases illustrate common scenarios and how to address them. Best practices and troubleshooting guidance help operators maintain the system effectively.

Capítulo 2

Technical Architecture

2.1 System Architecture Overview

The NeuralForgeAI system implements a microservices architecture with well-defined responsibilities for each component. The architecture is designed around a message broker (Redis) that coordinates communication between the various services, and a database (PostgreSQL) that stores Optuna study data and provides persistence for the distributed optimization process.



Figure 2.1: System Architecture Overview

The flow of a typical training job begins when a user submits a configuration file through either the REST API, Gradio interface, or React frontend. The Control Server validates the configuration and dispatches a task to the Manager component through the Redis message broker. The Manager creates or resumes an Optuna study and begins the optimization process, suggesting hyperparameters for each trial and dispatching training tasks to available workers. Each Invoker worker receives training tasks, creates isolated Docker containers for execution, and collects results upon completion. The training container performs the actual YOLO training, logs metrics, and returns accuracy results to the Optuna study. This process continues until all trials are completed or the study is terminated.

2.2 Component Architecture

2.2.1 Control Server

The Control Server serves as the primary entry point to the NeuralForgeAI cluster. Built on FastAPI, it provides a RESTful interface for programmatic access while also hosting a Gradio-based user interface for visual interaction. The component is responsible for configuration validation, priority mapping, and task dispatching to the Manager component.

The REST API exposes the following primary endpoints: POST /train for launching new training studies, GET /workers for listing available workers, GET /status/{study_id} for checking study status, GET /tasks for listing queued tasks, DELETE /tasks/{task_id} for revoking tasks, and POST /tasks/{task_id}/requeue for reprioritizing tasks.

The Gradio interface provides a user-friendly way to interact with the system without requiring API integration. Users can upload YAML configuration files, select priority levels and execution modes, and monitor study progress through the interface.

2.2.2 Manager (Optuna Orchestrator)

The Manager component is the orchestration brain of the system. It runs as a Celery worker that consumes tasks from the managers queue and coordinates Optuna studies. The component implements the following core functionality:

Study Management: Creates new Optuna studies or resumes existing ones based on the study name provided in the configuration. The study configuration includes the optimization direction (maximize or minimize), number of trials, and the search space definition.

Hyperparameter Suggestion: For each trial, Optuna suggests hyperparameter values based on the configured search space. The Manager supports various suggestion types including categorical choices, uniform distributions, log-uniform distributions, and integer ranges.

Task Dispatch: After determining the hyperparameters for a trial, the Manager dispatches a training task to the appropriate worker queue. The queue selection is based on the priority specified by the user and the target worker configuration.

Result Collection: Upon task completion, the Manager collects the accuracy metric from the training execution and reports it back to the Optuna study. This feedback loop allows Optuna to guide subsequent suggestions toward better configurations.

The Manager uses the SQLite storage backend for local study persistence, but the architecture supports PostgreSQL for distributed scenarios where multiple Managers coordinate on the same study.

2.2.3 Invoker (Celery Worker)

The Invoker is a Celery worker that listens on multiple queues for training tasks. Its primary responsibility is to manage the execution of training trials in isolated Docker containers. The Invoker implements a pipeline architecture with three main stages:

Pre-Train Stage (EDA): Before launching training, the Invoker can execute exploratory data analysis on the dataset to understand its characteristics and potential issues.

Training Stage: The core functionality involves receiving trial parameters, creating a temporary workspace, launching the Docker executor container, and monitoring execution. The Invoker uses the Docker SDK to programmatically manage container lifecycle, including

resource allocation (CPU and memory limits), GPU device passing, and volume mounting.

Post-Train Stage (LLM Analysis): After training completion, optional analysis can be performed using Large Language Models to provide insights about the training process and results.

The Invoker implements priority-based queue consumption, listening to queues in a strict order: private worker queues first (highest priority), then `gpus_high`, `gpus_medium`, and `gpus_low`. This ensures that critical tasks are processed before less important ones.

2.2.4 Worker Executor (Docker Container)

The Worker Executor is an ephemeral Docker container that performs the actual YOLO model training. Launched by the Invoker for each training trial, it receives configuration through a shared volume, executes the training process, and writes results back to the volume for collection by the Invoker.

The executor reads the training configuration from `config.json` in the mounted volume, loads the specified YOLO model, executes training with the provided hyperparameters, logs metrics to MLflow for tracking, and writes results (including accuracy) to `results.json` in the shared volume. The container is configured with auto-remove enabled, ensuring cleanup after each execution.

The executor supports various YOLO versions and model architectures, configured through the model parameter in the training configuration. It can handle classification, detection, and segmentation tasks.

2.2.5 NeuralForgeAI Frontend

The NeuralForgeAI frontend is a React application that provides a modern, intuitive interface for interacting with the training cluster. Built with TypeScript and Tailwind CSS, it offers a comprehensive set of features including project management, user management, training launch, service monitoring, and settings configuration.

The frontend communicates with the Control Server API to submit training jobs, retrieve worker status, and monitor study progress. It implements proper state management for handling user sessions, project selections, and configuration uploads.

2.3 Data Flow and Message Passing

The system uses Redis as the message broker for inter-component communication. Celery tasks are dispatched between components through designated queues:

The managers queue carries high-level study orchestration tasks from the Control Server to the Manager component. Worker queues (`worker_gpu1`, `worker_gpu2`, etc.) carry private, machine-specific training tasks. Public queues (`gpus_high`, `gpus_medium`, `gpus_low`) carry priority-ranked public training tasks.

This queue architecture enables both strict priority handling and the ability to direct specific jobs to particular workers when needed. The priority mapping ensures that critical training jobs are processed before background or experimental jobs.

2.4 Database Schema

PostgreSQL serves as the central data store for Optuna studies. Each study maintains metadata including the study name, creation timestamp, optimization direction, and the

set of completed trials. Each trial records the suggested hyperparameters, the evaluated metric value, the trial state (complete, pruned, or failed), and associated timestamps.

The database also stores worker metadata including worker identifiers, queue subscriptions, and last heartbeat timestamps. This information is used by the Control Server to provide accurate worker availability information to users.

Capítulo 3

Installation Guide

3.1 System Requirements

The NeuralForgeAI cluster has specific hardware and software requirements for proper operation. Each component has different requirements based on its role in the architecture.

The Control Server and Manager components require a system with at least 2 CPU cores, 4GB RAM, and network connectivity to Redis and PostgreSQL. These components can run on standard virtual machines or containers.

GPU Worker nodes require NVIDIA GPU with CUDA support, NVIDIA Docker runtime installed, and network connectivity to Redis and PostgreSQL. The number of workers depends on available GPU resources.

The Redis broker requires at least 2GB RAM for message queuing and should be deployed on a reliable network-attached system. Redis Enterprise or Redis Sentinel is recommended for production deployments.

PostgreSQL requires at least 4GB RAM and appropriate storage for study metadata. For distributed scenarios with multiple Managers, PostgreSQL is required instead of SQLite.

3.2 Infrastructure Setup

The following services must be running before deploying the application components:

Redis serves as the message broker and should be accessible at a known URL (e.g., `redis://192.168.10.252:23437/0`). PostgreSQL stores Optuna study data and should be accessible at a known URL (e.g., `postgres://postgres:postgres@192.168.10.252:23436/wyoloservice`). MLflow can be optionally deployed for experiment tracking and should be accessible at a known URL.

These services can be deployed using Docker Compose, Kubernetes, or any other container orchestration platform. The connection URLs must be configured in all application components.

3.3 Control Server Deployment

The Control Server can be deployed using Docker Compose:

```
1 cd wyoloservice2_control_server
2 docker-compose up -d
```

Listing 3.1: Deploy Control Server

The Control Server exposes two ports: port 8000 for the FastAPI REST API and port 7860 for the Gradio interface. After deployment, verify access to both endpoints.

3.4 Manager Deployment

The Manager component runs as a Celery worker and requires connection to Redis and PostgreSQL:

```
1 cd wyoloservice2_manager
2 docker-compose -f docker-compose.manager.yml up -d
```

Listing 3.2: Deploy Manager

The Manager uses the pool type solo (-P solo) to prevent deadlock when waiting for task results. This configuration ensures that each study runs sequentially through trials.

3.5 Worker Deployment

GPU Workers are deployed using Docker Compose with the appropriate environment variables:

```
1 cd wyoloservice2_invoker
2 WORKER_NAME=gpu_node_01 docker-compose up -d
```

Listing 3.3: Deploy GPU Worker

Each worker must have Docker installed and access to the GPU through NVIDIA Docker runtime. The worker subscribes to queues based on the PRIVATE_QUEUE environment variable.

Multiple workers can be deployed on the same machine by running different containers with different worker names, or on different machines by pointing to the same Redis broker.

3.6 NeuralForgeAI Frontend Deployment

The React frontend can be deployed for production using Docker:

```
1 cd NeuralForgeAI
2 docker build -t neuralforgeai:v1.0.0 .
3 docker run -p 3000:3000 -e
  REACT_APP_API_URL=http://control_server:8000
  neuralforgeai:v1.0.0
```

Listing 3.4: Deploy NeuralForgeAI

For development, the frontend can be run locally:

```
1 cd NeuralForgeAI
2 npm install
3 npm run dev
```

Listing 3.5: Frontend Development

Capítulo 4

Configuration Guide

4.1 Configuration File Structure

Training configurations are specified in YAML format. The configuration file contains several sections that define the training parameters and optimization settings:

```

1  # Configuration File: config_train.yaml
2  debug: "192.168.1.84"
3  type: "yolo"
4
5  # Base training configuration
6  train:
7    batch: 0.85
8    data: /dataset/
9    epochs: 250
10   imgsz: 640
11   cos_lr: true
12   lr0: 0.01
13   lrf: 0.01
14   dropout: 0.0
15   device: 0
16   workers: 8
17
18  # Optuna Sweeper Configuration
19  sweeper:
20    version: 1
21    study_name: "FirstClassifier"
22    direction: maximize
23    fitness: "metrics/accuracy_top1"
24    n_trials: 15
25    sampler: "TPESampler"
26
27  # Search Space Definition
28  search_space:
29    model: ["choice", "yolov8n-cls.pt", "yolo11s-cls.pt",
30            "yolo11n-cls.pt"]
31    train:
32      imgsz: ["choice", 480, 640]
33      epochs: ["choice", 150, 200, 300]
34      lr0: ["loguniform", 1e-4, 1e-2]
35      lrf: ["choice", 0.01, 0.2]
36      weight_decay: ["loguniform", 1e-5, 1e-3]
37      dropout: ["choice", 0.0, 0.1, 0.25]
38      mixup: ["choice", 0.0, 0.1, 0.2]
39      cos_lr: ["choice", true, false]
40
41  # Metadata
42  metadata:
43    author: "William Rodriguez"
44    content: "Training configuration for classifier"

```

Listing 4.1: Example Training Configuration

4.2 Configuration Parameters

The configuration file supports the following sections and parameters:

The debug section contains infrastructure configuration including server IP addresses and connection URLs for Redis and PostgreSQL.

The model section specifies the base YOLO model to use for training. Supported values include various YOLOv8 and YOLO11 variants for classification, detection, and segmentation tasks.

The train section defines the base training parameters. These values serve as defaults and can be overridden by the sweeper during optimization. Parameters include batch size, dataset path, number of epochs, image size, learning rate (lr0), final learning rate (lrf), dropout, data augmentation settings, device selection, and worker threads.

The sweeper section configures the Optuna optimization study. Key parameters include the study name (used for resuming studies), optimization direction (maximize or minimize), fitness metric to optimize, number of trials, sampler algorithm (TPESampler, RandomSampler, CmaEsSampler), and the search space definition.

The search space defines the hyperparameter ranges to explore. Each parameter supports different distribution types: choice for categorical selection, uniform for continuous uniform distribution, loguniform for logarithmic uniform distribution, and range for integer ranges with optional step values.

The metadata section provides information about the training configuration including author name, project description, and documentation.

4.3 Worker Configuration

Each Invoker worker is configured through a YAML file:

```
1  # invoker/config.yaml
2  redis:
3    host: "192.168.1.137"
4    port: 6379
5    db: 0
6
7  optuna:
8    storage_url:
9      "postgres://postgres:postgres@192.168.10.252:23436/wyoloservice"
10
11 celery:
12   concurrency: 1
13   loglevel: "INFO"
14   queue: "gpus"
15
16 worker:
17   executor_image: "wisrovi/train_service:worker_executor_v1.0.0"
18   host_temp_dir: "/tmp"
19   cpu_limit_pct: 0.85
20   mem_limit_pct: 0.60
```

Listing 4.2: Worker Configuration

The worker configuration defines Redis connection parameters, Optuna database URL, Celery settings, and resource limits for training containers.

Capítulo 5

Usage Examples

5.1 Launching a Training Study via API

The following example demonstrates how to launch a training study using the REST API:

```

1  import requests
2
3  # Prepare the configuration file
4  files = {'config_file': open('config_train.yaml', 'rb')}
5
6  # Specify execution parameters
7  data = {
8      'mode': 'public',      # or 'private'
9      'priority': 'high',    # high, medium, or low
10     'worker_name': ''      # required for private mode
11 }
12
13 # Submit the training job
14 response = requests.post(
15     "http://localhost:8000/train",
16     files=files,
17     data=data
18 )
19
20 # Get the study ID
21 result = response.json()
22 print(f"Study ID: {result['study_id']}")
23 print(f"Status: {result['status']}")
24 print(f"Queue: {result['worker_queue']}")

```

Listing 5.1: Launch Training via API

The response includes a study ID that can be used to monitor progress:

```

1  # Check study status
2  study_id = "your-study-id"
3  response =
4      requests.get(f"http://localhost:8000/status/{study_id}")
5  status = response.json()
6
7  print(f"State: {status['state']}")
8  if status['result']:
9      print(f"Best Parameters:
10          {status['result'].get('best_params')}")
11      print(f"Best Value: {status['result'].get('best_value')}")

```

Listing 5.2: Check Study Status

5.2 Launching a Training Study via Gradio

The Gradio interface provides a visual way to launch training studies:

Navigate to `http://localhost:7860` in a web browser. Select the New Study tab. Upload your configuration YAML file. Choose the execution mode (public or private). Select the priority level for public mode. If private mode, select the target worker. Click Launch Study to submit. The interface displays the Study ID for tracking.

5.3 Launching a Training Study via NeuralForgeAI

The React frontend offers the most comprehensive user experience:

Navigate to `http://localhost:3000`. Log in or select a user profile. Choose or create a project. Navigate to Launch Training. Upload the configuration YAML. The system validates the file automatically. Click Launch Training Job to submit. Monitor progress through the dashboard.

5.4 Querying Available Workers

To determine which workers are available for private mode execution:

```
1 response = requests.get("http://localhost:8000/workers")
2 workers = response.json()
3 print("Available Workers:")
4 for worker in workers:
5     print(f"    - {worker}")
```

Listing 5.3: List Available Workers

5.5 Managing Queued Tasks

View all tasks currently queued in the system:

```
1 response = requests.get("http://localhost:8000/tasks")
2 tasks = response.json()
3 print(f"Total Tasks: {tasks['count']}")
4 for task in tasks['queued_tasks']:
5     print(f"    - {task['task_id']}: {task['state']}")
```

Listing 5.4: List Queued Tasks

Revoke a specific task:

```
1 task_id = "task-id-to-revoke"
2 response =
3     requests.delete(f"http://localhost:8000/tasks/{task_id}")
4 print(response.json())
```

Listing 5.5: Revoke Task

Reprioritize a task:

```
1 task_id = "task-id-to-reprioritize"
```

```
2 response = requests.post(  
3     f"http://localhost:8000/tasks/{task_id}/requeue?priority=high"  
4 )  
5 print(response.json())
```

Listing 5.6: Reprioritize Task

Capítulo 6

Practical Use Cases

6.1 Use Case 1: Image Classification Model Training

A user wants to train a YOLO classification model to classify product images into multiple categories. The dataset contains 10,000 images across 20 product categories.

Configuration steps include setting model to yolov8n-cls.pt or yolo11s-cls.pt, specifying the dataset path in train.data, setting initial epochs to 50 for baseline, setting imgsz to 640 for standard resolution, defining search space for learning rate, image size, dropout, and augmentation parameters, setting n_trials to 20 for reasonable optimization, and setting direction to maximize for accuracy metric.

Expected results include optimized hyperparameters achieving 85-95% accuracy on the validation set, identification of optimal image size balancing accuracy and speed, and best data augmentation strategy for the specific dataset.

6.2 Use Case 2: Object Detection for Autonomous Vehicles

A company needs a high-accuracy object detection model for autonomous vehicle applications. The model must detect pedestrians, vehicles, traffic signs, and other objects in real-time.

Configuration approach uses detection model variants (yolov8n.pt, yolov8s.pt), larger image sizes (1280 or 1536), longer training (500+ epochs), extensive augmentation for robustness, and focus on recall metric alongside precision.

The sweeper configuration explores model sizes for speed-accuracy tradeoffs, input resolutions, anchor configurations, IoU thresholds, and NMS settings. Use case-specific metrics focus on mAP@0.5 and mAP@0.5:0.95.

6.3 Use Case 3: Industrial Quality Inspection

A manufacturing company needs to deploy a model for defect detection on production lines. The model must achieve very high precision to minimize false positives that cause unnecessary line stoppages.

Configuration strategy uses specialized detection models, high precision focus, balanced training with validation on production samples, and extensive data augmentation with lighting variations.

Optimization focuses on precision and F1 score, explores confidence thresholds, uses larger search trials for production quality, and includes fallback configurations for memory constraints.

Capítulo 7

Metrics and Performance

7.1 System Metrics

The NeuralForgeAI system provides several metrics for monitoring performance:

Training metrics include accuracy (classification top-1 accuracy), loss (training and validation loss curves), mAP (mean Average Precision for detection), and inference speed (FPS on target hardware).

System metrics include trial duration (time per training trial), queue wait time (time tasks spend waiting for workers), worker utilization (percentage of time workers are active), and study completion time (total time for full optimization).

Optimization metrics include best value (optimal metric achieved), convergence rate (trials needed for quality results), hyperparameter importance (which parameters matter most), and study comparison (comparing different study configurations).

7.2 Expected Performance Characteristics

The following performance characteristics can be expected under normal operating conditions:

Study completion time depends on the number of trials and trial duration. A 20-trial study with 100-epoch training typically completes in 10-40 hours depending on available GPU resources. Parallel trials reduce total completion time linearly with the number of workers.

Worker utilization typically achieves 80-95% when sufficient tasks are queued. Low utilization indicates insufficient workers or resource contention.

Optimization efficiency with Optuna's TPE sampler typically achieves good results within 10-15 trials for problems with fewer than 10 hyperparameters. Complex problems may require more trials.

7.3 Monitoring Tools

The system integrates with several monitoring tools:

MLflow provides experiment tracking with metrics logging, parameter tracking, artifact storage, and model versioning.

Optuna Dashboard offers study visualization, trial comparison, hyperparameter importance analysis, and optimization progress plots.

Flower (Celery Monitor) shows worker status, task queue depths, task execution times, and worker heartbeat status.

Capítulo 8

Best Practices and Troubleshooting

8.1 Deployment Best Practices

When deploying the NeuralForgeAI cluster, consider the following best practices:

Infrastructure deployment should use dedicated Redis and PostgreSQL instances for production, implement network security between components, configure proper resource limits for each service, and use container orchestration for high availability.

Configuration management should version control all configuration files, use environment-specific configurations (dev, staging, prod), validate configurations before submission, and document custom search spaces and their rationale.

Resource allocation should size GPU workers appropriately for expected workload, allocate sufficient CPU and memory to supporting services, use resource limits to prevent individual jobs from monopolizing resources, and monitor resource utilization and adjust accordingly.

Operational procedures should implement regular backup of Optuna database, establish monitoring and alerting for component health, create runbooks for common operational tasks, and schedule maintenance windows for updates.

8.2 Common Issues and Solutions

Issue: Tasks stuck in queue with workers available.

Possible causes include worker not subscribed to the correct queue, Redis connection issues, or worker process hang. Solutions involve verifying worker queue subscription, checking Redis connectivity, and restarting the worker process.

Issue: Training jobs failing with out-of-memory errors.

Possible causes include batch size too large for GPU memory, model size exceeding available memory, or image size too large. Solutions include enabling fallback configurations with reduced batch sizes, starting with smaller models, and using smaller image sizes.

Issue: Optuna study not improving over baseline.

Possible causes include search space too narrow, too few trials, or incorrect metric tracking. Solutions involve widening search space, increasing trial count, verifying metric names, and checking for trial failures.

Issue: Workers not appearing in worker list.

Possible causes include worker not started, wrong Redis URL configured, or private queue name not starting with `worker_`. Solutions involve checking worker logs, verifying Redis configuration, and confirming queue naming.

Issue: Long study completion times.

Possible causes include insufficient workers, trial duration too long, or excessive epochs. Solutions involve adding more workers, reducing trial epochs for exploration, and using early stopping.

Capítulo 9

Conclusions and Future Work

9.1 Conclusions

The NeuralForgeAI Distributed YOLO Training Cluster provides a comprehensive solution for distributed machine learning workloads. The architecture successfully addresses the challenges of scalability, resource management, and hyperparameter optimization that organizations face when training deep learning models.

Key achievements of the system include effective distributed orchestration using Celery and Redis for reliable task distribution, automated hyperparameter optimization with Optuna for efficient search, complete isolation through Docker containerization for reproducible training, multiple interface options including REST, Gradio, and React for different user preferences, and priority-based scheduling for optimal resource utilization.

The system has been designed with production deployment in mind, incorporating features such as resource limits, monitoring integration, and operational tooling. The documentation provided in this document enables operators to deploy, configure, and maintain the system effectively.

9.2 Future Work

Several enhancements are planned for future versions:

Scaling improvements will add support for Kubernetes deployment for better resource management, implement auto-scaling based on workload, and add multi-cluster support for large organizations.

Feature enhancements will integrate Ray Tune as an alternative optimizer, add support for additional model architectures beyond YOLO, implement transfer learning from pre-trained models, and add federated learning capabilities.

Operational improvements will enhance monitoring with custom dashboards, add automated testing and CI/CD pipelines, implement disaster recovery procedures, and add cost optimization features.

User experience improvements will expand the React frontend with more features, add collaborative features for team projects, implement model versioning and registry, and add API rate limiting and quotas.

Bibliography

bibitemcelery Celery: Distributed Task Queue. <https://docs.celeryproject.org/> bibitemoptuna Optuna: A Hyperparameter Optimization Framework. <https://optuna.org/> bibitemdocker Docker: Container Platform. <https://www.docker.com/> bibitemredis Redis: In-Memory Data Structure Store. <https://redis.io/> bibitempostgresql PostgreSQL: Relational Database. <https://www.postgresql.org/> bibitemyolo YOLO: You Only Look Once. <https://docs.ultralytics.com/> bibitemmlflow MLflow: Machine Learning Lifecycle Platform. <https://mlflow.org/> bibitemfastapi FastAPI: Modern Python Web Framework. <https://fastapi.tiangolo.com/> bibitemgradio Gradio: Machine Learning Web UIs. <https://gradio.app/> bibitemreact React: JavaScript Library for Building User Interfaces. <https://reactjs.org/> bibitemkub Kubernetes: Container Orchestration. <https://kubernetes.io/> bibitemnvidia NVIDIA Docker: GPU Container Runtime. <https://github.com/NVIDIA/nvidia-docker>

Appendix A

Advanced Configurations

A.1 Advanced Search Space Configurations

Advanced users can define complex search spaces with conditional parameters:

```

1  sweeper:
2    search_space:
3      # Conditional: model selection affects available parameters
4      model: ["choice", "yolov8n.pt", "yolov8s.pt", "yolov8m.pt"]
5
6      # Nested parameters
7      train:
8        # Conditional on model size
9        lr0: ["loguniform", 1e-5, 1e-2]
10       batch: ["choice", 8, 16, 32]
11
12       # Image size options
13       imgsz: ["choice", 416, 512, 640, 1280]
14
15       # Augmentation parameters
16       mixup: ["choice", 0.0, 0.1, 0.2, 0.3]
17       copy\paste: ["choice", 0.0, 0.1]
18
19       # Training strategy
20       close\_mosaic: ["choice", 10, 15, 20]
21       optimizer: ["choice", "SGD", "Adam", "AdamW"]

```

Listing A.1: Advanced Search Space

A.2 Docker Resource Limits

Advanced Docker configurations for production deployments:

```

1  worker:
2    executor_image: "wisrovi/train_service:worker_executor_v1.0.0"
3    host_temp_dir: "/tmp"
4
5    # Resource limits (percentages of host)
6    cpu_limit_pct: 0.85      # Use 85% of available CPU cores
7    mem_limit_pct: 0.60     # Use 60% of available RAM
8
9    # GPU configuration
10   gpu_devices: "0"         # Specific GPU device IDs
11   gpu\_utilization: 95     # Target GPU utilization
12
13   # Container runtime
14   shm\_size: "16g"         # Shared memory size
15   ulimits:
16     memlock: -1
17     stack: 67108864

```

Listing A.2: Advanced Docker Configuration

A.3 Multi-Cluster Deployment

For large organizations requiring multiple clusters:

1. Deploy shared Redis and PostgreSQL accessible from all clusters
2. Configure each cluster with unique worker prefixes
3. Use DNS or load balancers for API access
4. Implement cross-cluster job submission
5. Monitor resource usage across clusters

Appendix B

API Reference

B.1 REST API Endpoints

POST /train

Launch a new training study.

Request: multipart/form-data with config_file (YAML), mode (string), worker_name (string, optional), priority (string).

Response: JSON with status, study_id, mode, worker_queue.

GET /workers

List active private workers.

Response: JSON array of worker names.

GET /status/{study_id}

Get study status and results.

Response: JSON with study_id, state, result.

GET /tasks

List queued tasks.

Response: JSON with queued_tasks array and count.

DELETE /tasks/{task_id}

Revoke a task.

Response: JSON with status and task_id.

POST /tasks/{task_id}/requeue

Reprioritize a task.

Request: query parameter priority (high, medium, low).

Response: JSON with original_task_id, new_task_id, new_priority.

Appendix C

Acknowledgments

Acknowledgments

I would like to express my sincere gratitude to the open-source community for the exceptional tools that make this project possible: Ultralytics for the remarkable YOLO implementations, the Celery team for the robust distributed task queue, Optuna contributors for the powerful optimization framework, the Docker and Kubernetes communities for containerization technologies, FastAPI and Gradio teams for excellent Python web frameworks, and the React team for the modern UI library. I am particularly grateful to my colleagues at eCaptureDtech for their continuous support and feedback during the development of this system. Their practical insights have been invaluable in shaping the architecture and features. Special thanks to my family for their patience and understanding during the long hours dedicated to this project. Your encouragement has been the driving force behind this work. Finally, I would like to thank all users and contributors to the NeuralForgeAI project. Your engagement and feedback make this project meaningful and continuously improving.

William Rodriguez
Badajoz, Extremadura, España
March 17, 2026